

LAPORAN TUGAS 4
SISTEM OPERASI



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Mohamad Qowi Aizani (2410651003)

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH
JEMBER 2025

Tugas 1: Eksperimen dengan Data Berbeda

Data 1:

- First Come First Served

```
qowi@QOWIAIZANI:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time : 0
Burst Time   : 10

Proses P2
Arrival Time : 1
Burst Time   : 5

Proses P3
Arrival Time : 2
Burst Time   : 8

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
----
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time   : 7.33

=== GANTT CHART ===
| P1 | P2 | P3 |
10   15   23
```

- Shortest Job First

```
qowi@QOWIAIZANI:~$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time : 0
Burst Time   : 10

Proses P2
Arrival Time : 1
Burst Time   : 5

Proses P3
Arrival Time : 2
Burst Time   : 8

=== HASIL PENJADWALAN SJF ===
PID    AT    BT    CT    TAT    WT
----
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time   : 7.33

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P3
```

Data 2:

- First Come First Served

```
qowi@QOWIAIZANI:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time : 0
Burst Time   : 5

Proses P2
Arrival Time : 1
Burst Time   : 3

Proses P3
Arrival Time : 2
Burst Time   : 8

Proses P4
Arrival Time : 3
Burst Time   : 6

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
----    -
P1      0     5     5     5     0
P2      1     3     8     7     4
P3      2     8    16    14     6
P4      3     6    22    19    13

Average Turnaround Time: 11.25
Average Waiting Time   : 5.75

=== GANTT CHART ===
| P1 | P2 | P3 | P4 |
| 5  | 8  | 16 | 22 |
qowi@QOWIAIZANI:~$ |
```

- Shortest Job First

```
qowi@QOWIAIZANI:~$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time : 0
Burst Time   : 5

Proses P2
Arrival Time : 1
Burst Time   : 3

Proses P3
Arrival Time : 2
Burst Time   : 8

Proses P4
Arrival Time : 3
Burst Time   : 6

=== HASIL PENJADWALAN SJF ===
PID    AT    BT    CT    TAT    WT
-----
P1      0     5     5     5     0
P2      1     3     8     7     4
P3      2     8    22    20    12
P4      3     6    14    11     5

Average Turnaround Time: 10.75
Average Waiting Time   : 5.25

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P4 -> P3
```

Tugas 2: Analisis Eksperimen ketiga algoritma

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawaban:

1. Algoritma SJF (Shortest Job First) biasanya memberikan Average Waiting Time (WT) paling kecil.
Karena algoritma ini selalu mengeksekusi proses dengan waktu eksekusi paling singkat terlebih dahulu, sehingga proses-proses kecil tidak harus menunggu lama di antrian.
2. Karena setiap algoritma memiliki strategi prioritas berbeda dalam memilih proses berikutnya yang akan dijalankan.
3. Gunakan FCFS ketika:
 - Semua proses memiliki burst time yang hampir sama,
 - Tidak terlalu mementingkan efisiensi waktu tunggu,
 - Sistem sederhana, seperti batch processing atau antrian layanan satu arah.
4. Gunakan SJF ketika:
 - Burst time setiap proses diketahui sebelumnya,
 - Tujuan utama adalah meminimalkan waktu tunggu dan waktu rata-rata penyelesaian (TAT),
 - Sistem tidak terlalu membutuhkan keadilan antar proses.
5. Gunakan Round Robin ketika:
 - Sistem bersifat time-sharing atau multitasking,
 - Semua proses dianggap setara (fairness penting),
 - Cocok untuk sistem operasi interaktif (user banyak).

Tugas 3: Modifikasi Program

Tambahkan fitur berikut pada salah satu program:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

Langkah membuat 3 method tambahan pada fcfs:

```
#include <stdio.h>
#include <stdlib.h>

// Struktur data proses
struct Process {
    int pid; // ID Proses
    int arrival_time; // Waktu kedatangan
    int burst_time; // Waktu eksekusi
    int waiting_time; // Waktu menunggu
    int turnaround_time; // Waktu total (dari datang sampai selesai)
    int completion_time; // Waktu selesai
    int start_time; // Waktu mulai eksekusi
};

int main() {
    int n;
    FILE *inputFile, *outputFile;

    printf("=== SIMULASI ALGORITMA FCFS (MODIFIKASI) ===\n");

    // Membaca data dari file input
    inputFile = fopen("input_fcfs.txt", "r");
    if (inputFile == NULL) {
        printf("File input_fcfs.txt tidak ditemukan!\n");
        return 1;
    }

    fscanf(inputFile, "%d", &n); // Baca jumlah proses
    struct Process proc[n];

    for (int i = 0; i < n; i++) {
        fscanf(inputFile, "%d %d", &proc[i].arrival_time, &proc[i].burst_time);
        proc[i].pid = i + 1;
    }
    fclose(inputFile);

    // Hitung completion time, waiting time, turnaround time
    int current_time = 0;
    for (int i = 0; i < n; i++) {
        if (current_time < proc[i].arrival_time)
            current_time = proc[i].arrival_time;

        proc[i].start_time = current_time;
        proc[i].completion_time = current_time + proc[i].burst_time;
        proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;
        proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;

        current_time = proc[i].completion_time;
    }

    // Tampilkan hasil di layar
    printf("\n=== HASIL PENJADWALAN FCFS ===\n");
    printf("PID\tAT\tBT\tST\tCT\tTAT\tWT\n");
    printf("----\t---\t---\t---\t---\t---\t---\n");

    float total_tat = 0, total_wt = 0;
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            proc[i].pid,
            proc[i].arrival_time,
            proc[i].burst_time,
            proc[i].start_time,
            proc[i].completion_time,
            proc[i].turnaround_time,
            proc[i].waiting_time);

        total_tat += proc[i].turnaround_time;
        total_wt += proc[i].waiting_time;
    }

    printf("\nRata-rata Turnaround Time: %.2f\n", total_tat / n);
    printf("Rata-rata Waiting Time : %.2f\n", total_wt / n);

    // Gantt Chart lebih detail
    printf("\n=== GANTT CHART DETAIL ===\n");
    for (int i = 0; i < n; i++) {
        printf("| P%d ", proc[i].pid);
    }
    printf("\n");
}
```

```

printf("%d", proc[0].start_time);
for (int i = 0; i < n; i++) {
    printf("    %d", proc[i].completion_time);
}
printf("\n");

// Simpan hasil ke file output
outputFile = fopen("hasil_fcfs.txt", "w");
fprintf(outputFile, "=== HASIL PENJADWALAN FCFS ===\n");
fprintf(outputFile, "PID\tAT\tBT\tST\tCT\tTAT\tWT\n");
fprintf(outputFile, "----\t----\t----\t----\t----\t----\t----\n");

for (int i = 0; i < n; i++) {
    fprintf(outputFile, "%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        proc[i].pid,
        proc[i].arrival_time,
        proc[i].burst_time,
        proc[i].start_time,
        proc[i].completion_time,
        proc[i].turnaround_time,
        proc[i].waiting_time);
}

fprintf(outputFile, "\nRata-rata Turnaround Time: %.2f\n", total_tat / n);
fprintf(outputFile, "Rata-rata Waiting Time : %.2f\n", total_wt / n);

fprintf(outputFile, "\n=== GANTT CHART DETAIL ===\n");
for (int i = 0; i < n; i++) {
    fprintf(outputFile, "| P%d ", proc[i].pid);
}
fprintf(outputFile, "|\n");

fprintf(outputFile, "%d", proc[0].start_time);
for (int i = 0; i < n; i++) {
    fprintf(outputFile, "    %d", proc[i].completion_time);
}
fprintf(outputFile, "\n");

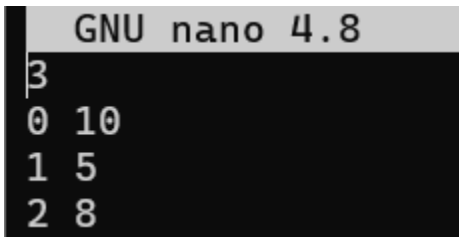
fclose(outputFile);

printf("\nHasil juga disimpan ke file 'hasil_fcfs.txt'\n");

return 0;
}

```

Setelah di compile, buat file input_fcfs.txt

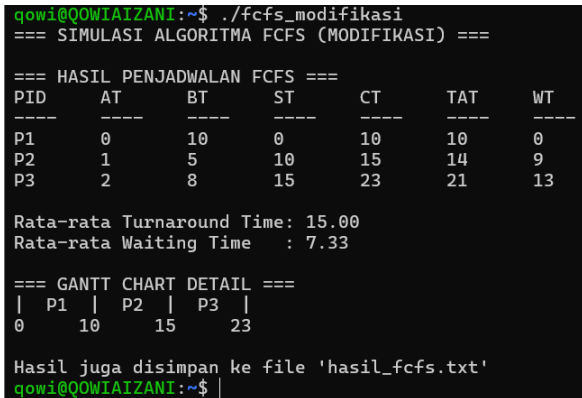


```

GNU nano 4.8
3
0 10
1 5
2 8

```

Maka outputnya akan menjadi seperti berikut:



```

qowi@QOWIAIZANI:~$ ./fcfs_modifikasi
=== SIMULASI ALGORITMA FCFS (MODIFIKASI) ===

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    ST    CT    TAT    WT
----
P1      0     10     0     10     10     0
P2      1      5     10     15     14     9
P3      2      8     15     23     21    13

Rata-rata Turnaround Time: 15.00
Rata-rata Waiting Time : 7.33

=== GANTT CHART DETAIL ===
| P1 | P2 | P3 |
0   10  15  23

Hasil juga disimpan ke file 'hasil_fcfs.txt'
qowi@QOWIAIZANI:~$

```

Tugas 4: Eksperimen dengan data berbeda untuk perbandingan algoritma

A. Proses dengan burst time bervariasi

```
qowi@QOWIAIZANI:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 3
Proses P1
  Arrival Time: 0
  Burst Time: 24

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 3

Proses P4
  Arrival Time: 3
  Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 21
Waktu 3: Menjalankan P1 -> Sisa waktu: 18
Waktu 6: Menjalankan P2 -> Selesai pada waktu 9
Waktu 9: Menjalankan P3 -> Selesai pada waktu 12
Waktu 12: Menjalankan P4 -> Sisa waktu: 3
Waktu 15: Menjalankan P1 -> Sisa waktu: 15
Waktu 18: Menjalankan P1 -> Sisa waktu: 12
Waktu 21: Menjalankan P1 -> Sisa waktu: 9
Waktu 24: Menjalankan P4 -> Selesai pada waktu 27
Waktu 27: Menjalankan P4 -> Selesai pada waktu 27

=== HASIL PENJADWALAN ROUND ROBIN ===


| PID | AT | BT | CT | TAT     | WT |   |
|-----|----|----|----|---------|----|---|
| P1  | 0  | 24 | 19 | 0514963 | 0  | 0 |
| P2  | 1  | 3  | 9  | 8       | 5  |   |
| P3  | 2  | 3  | 12 | 10      | 7  |   |
| P4  | 3  | 6  | 27 | 24      | 18 |   |


Average Turnaround Time: 10.50
Average Waiting Time: 7.50
```

B. Proses datang bersamaan

```
qowi@QOWIAIZANI:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 4
Proses P1
  Arrival Time: 0
  Burst Time: 10
Proses P2
  Arrival Time: 0
  Burst Time: 5
Proses P3
  Arrival Time: 0
  Burst Time: 8
Proses P4
  Arrival Time: 0
  Burst Time: 12
Proses P5
  Arrival Time: 0
  Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 6
Waktu 4: Menjalankan P2 -> Sisa waktu: 1
Waktu 8: Menjalankan P3 -> Sisa waktu: 4
Waktu 12: Menjalankan P4 -> Sisa waktu: 8
Waktu 16: Menjalankan P5 -> Sisa waktu: 2
Waktu 20: Menjalankan P1 -> Sisa waktu: 2
Waktu 24: Menjalankan P1 -> Selesai pada waktu 26
Waktu 26: Menjalankan P2 -> Selesai pada waktu 27
Waktu 27: Menjalankan P2 -> Selesai pada waktu 27
Waktu 27: Menjalankan P3 -> Selesai pada waktu 31
Waktu 31: Menjalankan P3 -> Selesai pada waktu 31

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
----
P1        0      10      26      26      16
P2        0       5      27      27      22
P3        0       8      31      31      23
P4        0      12     32766  -819627968  32766
P5        0       6     2096931418  0       0

Average Turnaround Time: -163925584.00
Average Waiting Time: 6565.40
```


C. Proses datang dengan interval

```
qowi@QOWIAIZANI:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 2
Proses P1
  Arrival Time: 0
  Burst Time: 8

Proses P2
  Arrival Time: 2
  Burst Time: 4

Proses P3
  Arrival Time: 4
  Burst Time: 9

Proses P4
  Arrival Time: 6
  Burst Time: 5

Proses P5
  Arrival Time: 8
  Burst Time: 3

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 6
Waktu 2: Menjalankan P1 -> Sisa waktu: 4
Waktu 4: Menjalankan P2 -> Sisa waktu: 2
Waktu 6: Menjalankan P1 -> Sisa waktu: 2
Waktu 8: Menjalankan P1 -> Selesai pada waktu 10
Waktu 10: Menjalankan P3 -> Sisa waktu: 7
Waktu 12: Menjalankan P1 -> Selesai pada waktu 12
Waktu 12: Menjalankan P2 -> Selesai pada waktu 14
Waktu 14: Menjalankan P4 -> Sisa waktu: 3
Waktu 16: Menjalankan P2 -> Selesai pada waktu 16
Waktu 16: Menjalankan P1 -> Selesai pada waktu 16

=== HASIL PENJADWALAN ROUND ROBIN ===
PID      AT      BT      CT      TAT      WT
---      --      --      --      ---      --
P1        0        8        16        16        8
P2        2        4        16        14        10
P3        4        9        1618075656    31487    -1655654752
P4        6        5        32764    292388368    32764
P5        8        3        1618072154      0      0

Average Turnaround Time: 58483976.00
Average Waiting Time: -331124416.00
```

Tugas 5: Analisis Eksperimen perbandingan algoritma

1. Analisis Performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?
FCFS (First Come First Served) memberikan hasil terbaik ketika semua proses memiliki waktu eksekusi (burst time) yang hampir sama dan tiba secara berurutan tanpa ada proses besar yang datang lebih awal. Dalam kondisi ini, waktu tunggu rata-rata menjadi kecil dan tidak ada proses yang terlalu lama menunggu.
- Mengapa SJF hampir selalu memberikan Average WT terkecil?
SJF (Shortest Job First) memprioritaskan proses dengan waktu eksekusi paling singkat. Karena proses singkat diselesaikan lebih dulu, maka waktu tunggu rata-rata (Average Waiting Time) berkurang secara signifikan. Secara teori, SJF adalah algoritma optimal untuk meminimalkan waiting time.
- Apa trade-off dari Round Robin?
Round Robin memberikan keadilan karena setiap proses mendapat jatah waktu (time quantum) yang sama. Namun, trade-off-nya adalah semakin kecil quantum, semakin sering terjadi context switch, yang menurunkan efisiensi CPU. Sebaliknya, quantum yang terlalu besar membuatnya mirip FCFS dan mengurangi responsivitas.

2. Context Switching

- Algoritma mana yang paling banyak melakukan context switch?
Round Robin adalah algoritma yang paling banyak melakukan context switch karena setiap proses hanya berjalan dalam jatah waktu (quantum) singkat sebelum digantikan proses lain.
- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?
Semakin kecil quantum → semakin banyak context switch.
Semakin besar quantum → semakin sedikit context switch.
- Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 — apa yang terjadi?
 - Quantum 2 → sangat responsif, tapi banyak context switch dan overhead tinggi.
 - Quantum 4 → masih responsif, sedikit lebih efisien.

- Quantum 8 → efisien, tapi mulai kehilangan interaktivitas.
- Quantum 16 → hampir seperti FCFS, waktu respons menurun, context switch sedikit.

3. Fairness

- Algoritma mana yang paling “adil” untuk semua proses?
Round Robin dianggap paling adil karena semua proses mendapatkan waktu CPU yang sama tanpa memandang besar kecilnya burst time.
- Proses mana yang paling dirugikan di algoritma FCFS?
Proses yang datang belakangan dengan burst time kecil akan sangat dirugikan jika di belakang proses besar (fenomena ini disebut convoy effect).
- Apakah ada kemungkinan starvation di SJF?
Ya. Proses dengan burst time panjang bisa kelaparan (starvation) jika selalu ada proses kecil yang datang lebih dulu dan terus diprioritaskan.

4. Real World Application

- Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?
Round Robin — karena memberikan keadilan dan waktu respons cepat untuk banyak permintaan kecil secara bergantian.
- Untuk render video (task besar), algoritma mana yang cocok?
FCFS atau SJF (Non-preemptive) — karena lebih efisien untuk proses besar dan panjang tanpa sering terhenti.
- Untuk OS desktop (interactive), algoritma mana yang cocok?
Round Robin (dengan quantum kecil) atau Multilevel Feedback Queue (MLFQ) — karena memberikan respons cepat untuk proses interaktif seperti klik, input keyboard, dan aplikasi ringan.